

Pruebas Unitarias - Módulo de Testing:

Implementamos las pruebas con la herramienta integrada de Django `django.test`, que está construida sobre `unittest`. Recordamos que las funciones testean la lógica y no tocan la base de datos aún. Es parte del esqueleto de la aplicación y su implementación completa se desarrollará en las siguientes semanas.

1. Pruebas de Lógica de Negocio (Lógica Pura) Utilizamos `SimpleTestCase` para evaluar las funciones de servicios (`services.py`) de manera aislada. Estas pruebas se ejecutan en memoria, no interactúan con la base de datos y verifican que la matemática de la aplicación sea exacta bajo diferentes escenarios. Hemos cubierto las siguientes funciones:

- `calculate_session_metrics`: Verificación de cálculo correcto de promedios y máximos de FPS, incluyendo el manejo de listas vacías.
- `toggle_overlay_state`: Validación del cambio de estado booleano de la interfaz sobrepuesta y manejo de excepciones por tipos de datos inválidos.
- `calculate_performance_gap`: Comprobación de las diferencias porcentuales de rendimiento respecto al mercado estándar, asegurando que no existan errores críticos como la división por cero.

2. Pruebas de Integración y API (Controladores) Haciendo uso de `TestCase` y el cliente de pruebas de Django, validamos el comportamiento de los endpoints principales (`views.py`). Aunque esta clase levanta una base de datos de prueba real (PostgreSQL), aplicamos técnicas de *Mocking* (`@patch` y `MagicMock`) para aislar los controladores y simular las interacciones con los modelos (`HardwareComponent` y `BenchmarkSession`). Las pruebas garantizan el cumplimiento de los requerimientos funcionales:

- **RF_11 (Componentes)**: Validamos la creación y eliminación de componentes, comprobando que los permisos de administrador (headers) bloqueen los accesos no autorizados (Errores 403).
- **RF_09 (Telemetría)**: Simulamos el envío de paquetes de datos anónimos, confirmando que se creen las sesiones correctamente y se rechacen los *payloads* incompletos (Errores 400).
- **RF_13 (Informes)**: Verificamos que la generación de reportes técnicos devuelva la estructura JSON correcta (métricas, línea de tiempo, hardware) y lance el código 404 al solicitar sesiones inexistentes.

```
(venv) estudiante@MonOrd:~/Ingesoft1-ProjectBackend/project-backend$ coverage run manage.py test core
Found 16 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 16 tests in 0.073s

OK
Destroying test database for alias 'default'...
```

```

destroying test database for alias 'default'...
(venv) estudiante@MonOrd1:~/ingesoft1-ProjectBackend/project-backend$ coverage report -m

```

Name	Stmts	Miss	Cover	Missing
benchmark_backend/__init__.py	0	0	100%	
benchmark_backend/settings.py	21	0	100%	
benchmark_backend/urls.py	4	0	100%	
core/__init__.py	0	0	100%	
core/admin.py	0	0	100%	
core/apps.py	4	0	100%	
core/migrations/0001_initial.py	7	0	100%	
core/migrations/0002_hardwarecomponent_benchmarksession_sessionsample.py	6	0	100%	
core/migrations/__init__.py	0	0	100%	
core/models.py	123	0	100%	
core/services.py	15	0	100%	
core/tests.py	165	0	100%	
core/views.py	137	54	61%	14, 23-26, 29-36, 41-45, 48, 51-57,
60-61, 66-72, 102-103, 130, 133-144, 149, 172-191, 255				
manage.py	11	2	82%	12-13
TOTAL	493	56	89%	

3. Pruebas de Lógica de Vistas (Lógica Interna) Se implementaron pruebas unitarias utilizando funciones auxiliares de las vistas (views.py), enfocadas en validar la lógica interna del sistema de forma aislada. Estas pruebas se ejecutan completamente en memoria, sin interacción con la base de datos, haciendo uso de técnicas de mocking (MagicMock y patch) para simular dependencias externas como modelos y consultas ORM. Las funcionalidades evaluadas fueron:

- `_is_admin`: Validación del reconocimiento de usuarios administradores mediante el análisis de headers en la petición, incluyendo el caso límite donde el header no está presente.
- `_require_admin`: Verificación del control de acceso a funcionalidades restringidas, asegurando que usuarios no autorizados reciban un error 403 y que los administradores puedan continuar sin restricciones.
- `_component_to_dict`: Comprobación de la correcta transformación de objetos de dominio en estructuras serializables (diccionarios), garantizando la integridad de los datos retornados al cliente.

```

(venv) PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> pytest core/test1.py
===== test session starts =====
platform win32 -- Python 3.12.4, pytest 9.1.0, pluggy 1.6.0
django: version: 6.0.6, settings: benchmark_backend.test_settings (from ini)
rootdir: C:\Users\sophi\Downloads\project-backend-main\project-backend-main
configfile: pytest.ini
plugins: django-4.12.0
collected 4 items

core/test1.py ....

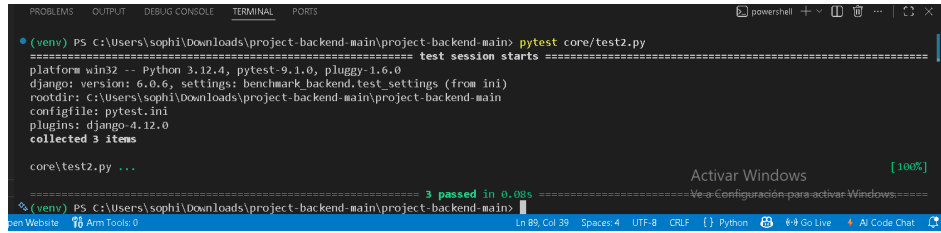
===== 4 passed in 0.25s =====

```

4. Pruebas de Procesamiento y Transformación de Datos Se desarrollaron pruebas unitarias orientadas al procesamiento de datos y generación de resultados dentro de las vistas, evaluando funciones críticas del sistema sin depender de la base de datos. Estas pruebas utilizan mocking para simular colecciones de datos y asegurar comportamientos determinísticos. Las funcionalidades evaluadas fueron:

- `_videogame_to_dict`: Validación de la correcta serialización de objetos de videojuegos, asegurando que los campos relevantes sean correctamente mapeados a estructuras JSON.
- `_recalculate_ranking`: Verificación del cálculo del promedio de puntuaciones de sesiones anónimas, asegurando la actualización correcta del atributo `base_score` del componente de hardware.

- `_recalculate_ranking` (caso límite): Evaluación del comportamiento cuando no existen sesiones disponibles, garantizando que no se realicen modificaciones ni operaciones innecesarias sobre los componentes.



```
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> pytest core/test2.py
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-9.1.0, pluggy-1.6.0
django: version: 6.0.6, settings: benchmark_backend.test_settings (from ini)
rootdir: C:\Users\sophi\Downloads\project-backend-main\project-backend-main
configfile: pytest.ini
plugins: django-4.12.0
collected 3 items

core/test2.py ...

===== 3 passed in 0.08s =====
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main>
```

The screenshot shows a Windows PowerShell terminal window with a dark background. The command `pytest core/test2.py` has been executed. The output displays the test session details, including the platform (win32), Python version (3.12.4), pytest version (9.1.0), pluggy version (1.6.0), Django version (6.0.6), and the settings used (benchmark_backend.test_settings). It also shows the root directory, config file (pytest.ini), and plugins (django-4.12.0). The test results indicate that 3 items were collected and all 3 tests passed in 0.08 seconds. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The status bar at the bottom shows the current file path, line and column numbers (Ln 89, Col 39), and various editor settings like Spaces (4), UTF-8, and CRLF.